

HealBridge — Final Project Report

Web Application and Defenses • Final consolidated implementation

1. Project overview

HealBridge is an academic Web Application and Defenses prototype for securely storing and explaining fictional prescription information. The project focuses on secure authentication, role-based access control, private document handling, server-side file validation, uncertainty handling, audit logging, and defenses mapped to the OWASP Top 10 (2025 taxonomy used by the project).

The application is intentionally presented as an academic prototype. It does not diagnose, prescribe, change medication instructions, or claim medical-grade OCR.

2. Project objectives

The implementation was developed around these objectives:

- provide registration and secure login;
- protect patient records with server-side authorization and ownership checks;
- allow controlled prescription-document uploads;
- validate uploaded files independently on the server;
- extract a limited structured representation from fictional samples;
- make uncertainty visible rather than silently presenting uncertain data as reliable;
- record security-relevant events;
- expose an ADMIN-only Security Center;
- map implementation controls to OWASP Top 10 categories;
- provide repeatable security and safety regression tests;
- provide a responsive, familiar dashboard with light and dark themes.

3. Technology stack

Frontend

- React
- React Router
- Vite
- Responsive CSS
- Light/dark theme persistence through browser local storage

Backend

- Python
- FastAPI
- SQLAlchemy
- Pydantic
- PyJWT
- pwdlib with Argon2id
- SlowAPI for rate limiting
- pypdf for controlled PDF parsing
- Pillow for image validation

Storage

- SQLite for the MVP database
- Private server-side upload directory for accepted files
- SHA-256 fingerprints for stored uploads

4. Architecture

Browser / React UI

|

| HTTP + Bearer token

v

FastAPI application

|

+--> Authentication / RBAC

+--> Prescription API

+--> File validation

+--> Structured demo extraction

+--> Audit logging

+--> Rate limiting

+--> OWASP Security Center

|

+-----+

||

v v

SQLite Private uploads

Trust boundaries

1. Browser → API: all client input is untrusted.
2. Upload → validation: extension, MIME, signature and parser checks are performed server-side.
3. User → prescription: view, download and delete operations perform ownership checks.
4. User → admin functions: the ADMIN role is enforced on the server.
5. Database/files → UI: only authorized API responses are returned.

5. Implementation history

Phase 1 — Authentication & Access Security

Implemented:

- registration with server-side validation;
- Argon2id password hashing;
- password policy: at least 10 characters plus uppercase, lowercase and numeric characters;
- generic invalid-login response;
- JWT bearer authentication with expiry;
- unique JWT identifiers (``jti``);
- server-side revoked-token records;
- logout token revocation;
- protected routes;

- PATIENT and ADMIN roles;
- server-side prescription ownership checks;
- cross-user access denial with `403`;
- unauthorized-access audit events;
- protected ADMIN endpoint;
- automated Phase 1 regression tests.

Phase 2 — File & Input Security

Implemented:

- allowlisted PDF, PNG and JPG/JPEG extensions;
- MIME validation;
- magic-byte/signature validation;
- parser-level PDF validation using pypdf;
- parser-level image validation using Pillow;
- empty-file rejection;
- 5 MB maximum upload size;
- incremental upload reading;
- 30-page PDF limit;
- 25-million-pixel image limit;
- safe filename normalization;
- random server-side storage filenames;
- storage path containment checks;
- SHA-256 file fingerprints;
- controlled malformed-input errors;
- SQLAlchemy parameterized database access;
- frontend raw-HTML injection avoidance;
- automated Phase 2 regression tests.

Phase 3 — OWASP & Backend Security

Implemented:

- trusted-host protection;
- restrictive CORS;
- bearer-token-compatible cross-origin defense-in-depth;
- security headers;
- optional HSTS for HTTPS deployments;
- generic validation errors;
- generic unexpected-error responses;
- `X-Request-ID` response correlation;
- `Retry-After` on rate-limit responses;
- dummy password-hash verification for unknown accounts to reduce timing differences;
- expired revoked-token cleanup;
- pinned direct dependencies;
- automated/static Phase 3 security checks.

Phase 4 — Security & Safety Testing

Implemented the final regression framework covering:

- unauthenticated access;
- IDOR / cross-user access;
- RBAC;
- forged tokens;
- logout and token replay;
- SQL-injection-style inputs;
- XSS-style input handling;
- hostile Origin requests;
- security headers;
- malicious/unsupported uploads;
- renamed executable uploads;
- malformed PDFs;
- oversized uploads;
- invalid resource identifiers;
- rate limiting;
- audit evidence;
- uncertainty/verification-required workflow;
- frontend raw HTML injection invariant.

The test runner is ``backend/test_phase4.py`` and uses an isolated temporary SQLite database.

Phase 5 — Finalization & Presentation

Implemented:

- familiar productivity-style dashboard;
- light/dark themes;
- responsive desktop/tablet/mobile layout;
- active navigation states;
- Prescription Vault;
- search and status filtering;
- drag-and-drop secure upload UI;
- prescription detail view;
- verification states;
- SHA-256 integrity display;
- secure original-file download;
- ADMIN Security Center;
- OWASP control map;
- audit trail;
- audit CSV export;
- architecture documentation;
- OWASP matrix;
- security test plan;

- test report template;
- final submission checklist;
- classroom demonstration script.

The dashboard does not contain a separate presentation-readiness panel.

6. Authentication and authorization

Registration

New accounts are created as `PATIENT`. There is no public role-selection field.

Password protection

Passwords are hashed using Argon2id through pwlib. Plaintext passwords are not stored.

JWT sessions

Authenticated requests use an expiring signed bearer token. Each token contains a unique `jti`. Logout records the `jti` in the revoked-token table, causing replay of the old token to return `401`.

RBAC

ADMIN-only endpoints require the server-side `ADMIN` role. Patients cannot promote themselves during registration.

Ownership

Prescription view, download and delete operations verify that the record belongs to the authenticated user. Cross-user attempts return `403` and are audited.

7. Secure upload architecture

Upload

```
|
+--> extension allowlist
|
+--> MIME check
|
+--> magic-byte check
|
+--> parser validation
|
+--> resource limits
|
+--> SHA-256
|
+--> random private storage name
|
+--> structured demo extraction
|
+--> uncertainty decision
v
```

Prescription record

The original client filename is retained only as a normalized display/download name. It is never used as the storage path.

8. Prescription extraction and safety

The prototype performs controlled structured extraction for fictional samples. PDF text is parsed when available, and a small catalog supports the supplied fictional sample files.

This is deliberately not described as clinical OCR. When required information is missing or the filename indicates an intentionally uncertain test case, the record is marked for verification.

The application explains existing prescription information only. It does not prescribe, diagnose or modify medication instructions.

9. OWASP Top 10 mapping

10. Security testing

The final test suite is designed to demonstrate defenses rather than merely describe them.

Core expected results

- unauthenticated protected route → `401`;
- cross-user prescription access → `403`;
- patient to ADMIN endpoint → `403`;
- forged token → `401`;
- logout followed by token replay → `401`;
- SQL injection attempt → no authentication bypass;
- XSS-style input → handled as data;
- hostile Origin → `403`;
- `.exe` upload → `400`;
- renamed executable `.pdf` → `400`;
- malformed PDF → `400`;
- oversized upload → `413`;
- rate limit exceeded → `429` with `Retry-After`;
- uncertain sample → verification required;
- unauthorized access → audit event.

Running all regression tests

```
python backend\test_phase1.py
```

```
python backend\test_phase2.py
```

```
python backend\test_phase3.py
```

```
python backend\test_phase4.py
```

Phase 1 and Phase 2 use isolated databases. Phase 3 performs static security configuration checks. Phase 4 uses an isolated database and exercises the application through FastAPI TestClient.

The runtime test results must be captured from the target Windows environment after dependencies are installed. The packaged source itself was syntax-checked during final assembly.

11. UI/UX features

The final interface provides:

- light theme;
- dark theme;
- persistent theme selection;
- responsive layout;

- sidebar navigation;
- dashboard metrics;
- recent activity;
- Prescription Vault search/filter;
- secure upload dropzone;
- clear validation messages;
- verification-required state;
- secure download action;
- ADMIN-only Security Center;
- audit-log export.

The interface intentionally avoids presenting the application as a clinical decision system.

12. Database design

Users

Stores identity, email, password hash, role and creation time.

Prescriptions

Stores ownership, extracted fields, explanation, verification state, original display filename, generated storage filename, SHA-256 fingerprint and creation time.

Revoked tokens

Stores JWT identifiers that have been revoked and their expiration information.

Audit logs

Stores security-relevant events, user ID where available, details, IP address and timestamp.

13. Test data

The project uses fictional academic prescription samples. They are intended only for testing upload, extraction and uncertainty workflows. They must not be treated as real medical records or medical advice.

14. Limitations

1. SQLite is used for the MVP instead of PostgreSQL.
2. Prescription extraction is controlled/demo extraction rather than medical-grade OCR.
3. The project is a local academic prototype, not a production healthcare deployment.
4. HTTPS/HSTS is deployment-dependent; HSTS is disabled by default for local HTTP development.
5. The bearer-token architecture does not use cookie authentication. If cookies are introduced later, a formal CSRF token mechanism must be added.
6. Security claims should be supported by actual runtime test evidence and screenshots in the final submission.

15. Future improvements

- PostgreSQL deployment configuration;
- stronger production secrets management;
- HTTPS reverse proxy and production HSTS configuration;
- real OCR with human-review safeguards;
- multilingual/Tamil localization;
- more advanced monitoring and alerting;

- external security assessment;
- production-grade object storage and malware scanning;
- stronger account recovery and device/session management.

16. Final conclusion

HealBridge now demonstrates a complete secure web-application workflow rather than only a prescription-upload interface. The implementation combines authentication, authorization, secure file handling, controlled extraction, uncertainty handling, audit logging, backend hardening, OWASP-oriented defenses, a polished responsive UI, and repeatable security/safety regression tests.

The project should be presented as an academic demonstration of web security engineering. Its strongest evidence is the combination of server-side controls and executable tests showing that unauthorized access, malformed files, oversized uploads, invalid sessions, hostile origins and other controlled attack cases are rejected as designed.